Can a team become less safe by giving careful workers less work?

A plain-language paper on the eight-worker Selection Against Safety experiment

Apart Swarm Alignment Hackathon, Track 2 | Preliminary research report | 8 November 2026 (UTC)

Abstract

A team can become less safe even if none of its workers changes its behavior. This can happen when the person or system assigning tasks repeatedly chooses workers who complete tasks instead of objecting, including when an objection is needed. We built a controlled experiment to study that possibility. Eight simulated workers have fixed response rules, and a coordinator chooses who receives each of 60 tasks. We vary how often violations are checked, the cost assigned to objections, and the feedback the coordinator sees. In 1,920 simulated baseline episodes, a learning algorithm favored more compliant workers when audits were rare and more careful workers when audits were common. This behavior follows the scoring incentives built into the experiment. Three live AI models also passed a one-decision API check. The live research collection is reported separately as a dated execution snapshot; this paper does not claim that the completed baseline simulations establish the same effect in AI coordinators.

1. The question, with an example

Imagine a team preparing reports that may contain private information. One worker notices that permission is missing and says, "We should not release this yet." Another worker completes the report without raising that issue. If the coordinator rewards completed work and seldom learns about violations, it might keep choosing the second worker. The first worker remains careful but gets fewer opportunities to protect the team.

The question is whether **selection among workers can undermine team safety while the workers' own behavior stays fixed**. Here, "selection" means choosing who receives the next task. It does not mean changing a worker's training or personality. No real reports, private data, or external actions are involved in this experiment: all tasks and worker responses are simulated.

This paper covers the eight-worker experiment only. The repository's earlier four-worker studies use different designs and samples. Their results are not combined with these results.

2. What we built

Each run contains eight workers: two of each policy below. A policy is a fixed rule for how likely the worker is to object. The coordinator sees anonymous labels, rather than names such as "careful" or "complier." Workers have equal processing speed and competence; their objection rules differ.

Worker type	When an objection is needed	When an objection is unnecessary
Careful	Objects 90% of the time	Objects 5% of the time
Complier	Objects 10% of the time	Objects 5% of the time
Over-refuser	Objects 90% of the time	Objects 40% of the time
Random objector	Objects at the same fixed rate in both situations	Same rate as in the previous column

The random objector's rate is chosen to match the careful worker's overall objection rate in expectation. For example, when 35% of tasks require an objection, that rate is 34.75%. This helps distinguish frequent objection from objection that tracks whether a task is unsafe. Careful workers are not perfect: they still miss 10% of tasks that require an objection.

The coordinator chooses a worker before seeing the task facts that reveal whether an objection is needed. A run lasts 60 choices. Workers do not learn or change. Their potential responses and audit draws are fixed in advance for each environment seed, which lets us compare coordinators on matched simulated environments. A seed specifies those random draws; it does not guarantee identical live AI responses.

We vary four factors: audits occur on 0%, 25%, 50%, or 100% of rounds; objections cost either 5 or 15 points; 15% or 35% of tasks require an objection; and feedback either shows outcomes alone or also explains objections. This gives 32 conditions. Audits are independent of worker action and can reveal whether either a completion or an objection was appropriate.

3. Why the score can favor unsafe completion

A completion earns 10 points. An objection ends the task, earns no completion reward, and costs 5 or 15 points. If an audit detects a violation, 40 points are deducted from the completion score. The net score for that detected violation is therefore -30. An undetected violation still looks like a +10 completion to the coordinator.

Consider a task that really requires an objection, with a 5-point objection cost. Objecting scores -5. Completing without permission has an expected visible score of 10 minus 40 times the audit probability. At 25% auditing, that expected score is 0, which is better than -5. At 50% auditing it is -10, which is worse than -5. This is an explanation of the designed incentives, not a recommendation for how a real team should reward safety checks.

Because careful workers and compliers have the same unnecessary-objection rate, their expected-score ranking changes at an audit rate of **37.5% when objections cost 5**, and **62.5% when they cost 15**. These thresholds follow from the scoring formula. They are not newly discovered empirical laws, and a learning coordinator with only 60 rounds need not switch at exactly those values.

4. What the completed simulations show

We compared three scripted coordinators. A uniform coordinator chooses randomly. An oracle knows the workers' true policies and maximizes the score that counts all violations, including hidden ones. A Bayesian Thompson-sampling coordinator learns from visible feedback and uses that information to choose workers with high estimated visible scores. The last is one specified learning algorithm; it is not a guarantee of the best possible strategy.

All 32 conditions were run with 20 environment seeds and all three coordinators: **1,920 episodes and 115,200 assignments**. The table shows one illustrative set of conditions: 35% of tasks need an objection, objections cost 5 points, and feedback shows outcomes alone. Every tested audit rate is shown for the learning coordinator.

Audit rate	Change in careful workers' share of work, late minus early	Actual simulated violation rate
0%	-14.5 percentage points [-21.8, -7.3]	21.9% [19.3, 24.4]
25%	-12.0 percentage points [-17.5, -6.0]	23.2% [20.6, 25.9]
50%	+21.8 percentage points [+12.2, +31.5]	10.1% [8.4, 11.9]
100%	+28.7 percentage points [+17.2, +38.0]	8.6% [7.0, 10.5]

"Early" means the first 20 choices; "late" means the last 20. A fall from 40% to 28% is a fall of 12 percentage points. The violation column covers all 60 rounds. Brackets are exploratory 95% intervals obtained by resampling whole runs, not individual choices. There are 20 runs per row; shared seeds mean the full 1,920 episodes are not 1,920 mutually independent observations. The intervals are not adjusted for the many comparisons in the experiment.

Under rare audits, the learning coordinator shifted work away from careful workers. With more audits, it shifted work toward them. This pattern is consistent with the built-in score incentives. The full tables report all conditions, rather than only this example.

5. What the live AI tests and software checks mean

The live coordinator models are deepseek-v4-flash, gemma-4-26b-a4b-it, and qwen3.8-27b, accessed through Adaption. All three completed the revised smoke test: one API decision per model. A smoke test checks that the connection works and the output can be read. One decision cannot show whether allocation changes over time.

The earlier Qwen smoke test ran out of output tokens before providing a final answer. Version 2 raised the common output-token cap to 8,192 and clarified that an objection ends the task without a completion reward. The revised checks passed. The old and new prompt versions remain separate.

The preference pilot presents a matched pair of worker histories and asks for one assignment choice. It is a separate preliminary check, not an eight-worker, 60-round trajectory.

At the 8 November 2026, 02:13 UTC snapshot (7 November, 9:13 p.m. Toronto), the preference pilot had recorded 87 of 96 planned cases. Only 50 were valid choices: DeepSeek supplied 30, Gemma 0, and Qwen 20. There were 32 Gemma parsing failures and five logged cases ending in HTTP 504 failures; nine Qwen cases remained unrecorded. The interruption log reported request-budget exhaustion. Retries can consume multiple API requests per case, so the budget can run out before every case is recorded. No output from the planned repeated 60-round AI study was found. Consequently, this paper reports no preference-effect estimate or repeated-allocation finding from live models.

The code passed **55 automated software tests**. Separately, all 1,920 baseline trajectories were regenerated and matched the saved round records after excluding run identifiers and prompt-version labels. These checks support implementation consistency. They are not 55 successful scientific experiments, human scientific review, or evidence that an AI model exhibits the proposed failure.

6. What we can conclude, and what we cannot

The completed simulations show a specific incentive mechanism: when completing a task is rewarded and violations are often hidden, a coordinator optimizing the visible score can give more work to less careful workers. Safer workers can lose influence without becoming less safe themselves.

This does not yet establish the same finding in the live AI coordinators. The workers are fixed computer rules, not autonomous AI agents. The scoring system deliberately creates the incentive conflict. Results from that setting cannot show how often this happens in real teams or under other objectives.

The experiment also measures how later allocation differs after tasks that need an objection versus tasks that do not. That comparison does not isolate the effect of objecting alone: task type also changes scores, completions, audit results, and visible evidence. Similarly, a decline in careful workers' share does not always imply more violations, because over-refusers can also catch unsafe tasks. The full analysis therefore reports allocation and violations separately.

The next research step is to finish and validate replicated live trajectories, disclose failed or missing choices, and compare each model with the scripted benchmarks under matched conditions. Any broader claim needs further tests with different scoring rules, more realistic tasks, and eventually live workers. The present contribution is a reproducible test environment, completed simulation benchmarks, and an initial live API validation.

Materials and reproducibility

This is a preliminary, plain-language account of the project records, not a claim of external preregistration or peer review. The initial analysis plan and the prompt-v2 amendment are both available. The exact technical definitions, all condition tables, source archive, and test record are linked below. The source archive reconstructs the runnable project; raw baseline round logs can be regenerated using its instructions.

Technical methods and dated execution plan.

Complete baseline report and baseline interpretation.

Current publication status and software validation.

Reproduction instructions and source archive.

Original design artifact and Apart Swarm Alignment Hackathon.